

Formatted Questions and Answers

Answer:

Yes, I am familiar with Ground News. It is a news aggregator platform and application designed to help readers analyze media bias and compare how different news outlets cover the same stories.

Its key features include:

- * **Bias Ratings:** Displaying the political leaning (Left, Center, Right) of the publications covering a specific story.
- * **The Blindspot:** Highlighting important news stories that are disproportionately covered or ignored by one side of the political spectrum.
- * **Factuality Scores:** Providing ratings on the historical reliability and factual accuracy of different news sources.

Are you looking to integrate a news API similar to it, or did you have a specific technical question related to Ground News?

Question 1:

im in my 8th sem and have to do a major project for it. and im thinking of do something same but in nepal's context.

Answer:

That is an excellent idea for an 8th-semester major project. A "Ground News for Nepal" is highly relevant, solves a real-world problem (misinformation and media bias), and demonstrates a strong mix of software engineering, data science, and web development skills.

To build this, you will need to break the project down into manageable technical components. Here is a high-level overview of how you can approach it, along with a suggested tech stack and potential challenges.

1. Core Architecture

You will need a pipeline that goes from raw news data to a processed, user-friendly interface.

- * **Data Ingestion (Scraping):** Automatically pull articles from major Nepali news outlets (e.g., OnlineKhabar, Kantipur, Setopati, The Kathmandu Post, Republica).
- * **Data Processing & Clustering:** Group articles from different publishers that are talking about the *exact same event/story*.
- * **Bias & Sentiment Analysis:** Analyze the article text to determine how the story is being framed.
- * **Database:** Store the articles, their sources, and the analysis results.
- * **Frontend Interface:** Display the clustered stories and their bias breakdowns (similar to Ground News' UI).

2. Suggested Tech Stack

- * **Backend & APIs:** Python (FastAPI or Django). Python is mandatory here because of its massive

ecosystem for scraping and Machine Learning.

- * **Web Scraping:** `BeautifulSoup`, `Scrapy`, or simply parsing RSS feeds (many Nepali news sites have RSS feeds, which makes things much easier).

- * **Database:** PostgreSQL (for structured relational data) or MongoDB (for storing unstructured article data).

- * **Machine Learning / NLP:**

- * **Clustering:** TF-IDF with K-Means, or using transformer embeddings to find similar articles.

- * **NLP:** HuggingFace `transformers` (multilingual models like XLM-RoBERTa handle Nepali text decently well).

- * **Frontend:** React, Next.js, or Vue.js. If you want a mobile app, Flutter is a great choice.

3. The "Nepal Context" Challenges (How to tackle them)

- * **Language Barrier:** Nepali NLP is not as advanced as English.

- * **Solution idea:** You might want to translate Nepali text to English using an API (like Google Translate or an open-source model) behind the scenes *just* for the ML processing step, while displaying the original Nepali text to the user. Alternatively, use Multilingual BERT models.

- * **Defining Bias:** In the US, it's mostly Left vs. Right. In Nepal, media bias often leans towards specific political parties (NC, UML, Maoist, RSP) or regional interests.

- * **Solution idea:** Instead of a simple Left/Center/Right scale, you might need to create a custom bias matrix or simply show "Sentiment toward [Entity/Politician]". You can manually assign a "Historical Leaning" score to certain news outlets based on known affiliations.

4. Minimum Viable Product (MVP) for your 8th Sem

If you try to do everything perfectly, you might run out of time. For a university project, I recommend starting small:

1. **Phase 1:** Pick just 4-5 major news sources (2 English, 3 Nepali).
2. **Phase 2:** Write scripts to scrape their RSS feeds every few hours.
3. **Phase 3:** Implement a basic clustering algorithm (e.g., if two articles from today share 5 of the same keywords/entities, group them together as one "Story").
4. **Phase 4:** Build a simple web frontend to display the grouped stories side-by-side.

How would you like to start?

I can help you design the system architecture, write the web scrapers for specific Nepali news sites, or help you set up the initial backend framework in this folder.

Question 2:

nice you got gist of it.

i like the part you understood that in nepal context we just can't label(left, right, center) simply coz let's be honest here's no one care about the ideology. Before diving into the tech and execution part let's first clear ourselves about what we will do about this.

Answer:

You are absolutely right. In Nepal, politics is largely driven by personalities, factions, and pragmatic alliances rather than strict ideologies like "Left" or "Right." A communist party (UML or Maoist) and a democratic party (NC) will happily form a coalition overnight! So, a traditional left/right spectrum

would be useless and confusing.

Let's step back and conceptualize the product. If we aren't measuring Left vs. Right, what exactly are we measuring, and what value are we providing to the user?

Here are a few alternative ways we could categorize and analyze news for the Nepali context. Let me know which of these make the most sense to you:

1. The "Establishment vs. Alternative" Metric

Right now, the most significant divide in Nepali politics and media is between the traditional power structures and new/independent forces.

- * **Pro-Establishment:** Media that tends to favor traditional parties (NC, UML, Maoists) or the status quo.
- * **Pro-Alternative:** Media that favors newly emerged parties (like RSP), independent figures (like Balen Shah, Harka Sampang), or disruptive politics.
- * **Neutral/Objective:** Focuses strictly on the facts without obvious framing.

2. Geopolitical Framing (The Foreign Policy Angle)

This is a huge factor in Nepali journalism. Many news portals have subtle (or not-so-subtle) biases regarding foreign relations, especially during major infrastructure or treaty debates (like the MCC or BRI). You could tag sources or articles based on their geopolitical framing:

- * Pro-India / Pro-China / Pro-West / Non-aligned.

3. Entity-Based Sentiment (The most data-driven approach)

Instead of classifying the news portal's overall bias, the app could analyze how a specific article treats a specific person or topic.

- * **Example:** "OnlineKhabar's coverage of this scandal is **Highly Critical** of Party X, whereas Kantipur's coverage is **Defensive**."
- * **Why it works:** You don't have to permanently label a news site. You just let the NLP model detect if the article is praising, criticizing, or neutrally reporting on a specific entity.

4. Sensationalism & Credibility Score (The "Kanda" Metric)

Nepal has a massive problem with clickbait, unverified online portals, and sensationalizing every minor event into a *kanda* (scandal).

- * **Mainstream Media (Broadsheets):** High verification, lower sensationalism.
- * **Tabloid/Online Portals:** High sensationalism, emotional language, clickbait headlines.
- * **Feature Idea:** A "Sensationalism Meter" that compares the headline to the actual content of the article to see if it's clickbait.

What should the Core Features be?

If we combine these ideas, the core value of your app could be:

- * **Headline Stacking:** Show 3 different headlines for the *exact same event* side-by-side so the user immediately sees the framing difference.
- * **The Overlooked/Hidden Stories:** Show which stories are getting massive coverage by mainstream media, but being ignored by alternative media (or vice versa).
- * **Credibility/ sensationalism tags:** Tagging articles as "Opinion", "Clickbait", "Verified Source",

or "Unverified Source."

What do you think of these angles? Should we focus on the **Establishment vs. Alternative** divide, focus purely on **Sensationalism/Clickbait**, or do you have another classification in mind?

Question 3:

i like some of these:

1. The "Establishment vs. Alternative" Metric
2. Geopolitical Framing. even this but not so much because this kind a rare not all topics get foreign involvement
3. Entity-Based Sentiment (The most data-driven approach). this we will do for sure
4. the one i have is what if we also focus on specific house favouring specific party or the political figure.

also if nepali data analysis is little though then what if we only do english sites only?. and would it be very hard to analyze and present results on these many parameters?

Answer:

These are great insights. Combining your idea (Tracking which media house favors which party/figure) with Entity-Based Sentiment is actually the most accurate way to map media bias in Nepal.

Let's address your two major questions, as they will define the scope of your 8th-semester project:

1. What if we only do English sites? (Is Nepali NLP too hard?)

Short Answer: Sticking to English sites for your MVP (Minimum Viable Product) is a very smart move.

Why?

- * **The Problem with Nepali NLP:** While tools exist, they require a lot of custom training. Performing Named Entity Recognition (NER) to find politicians' names, and then doing Sentiment Analysis on Nepali text, can easily turn into a research paper on its own, taking time away from building the actual app.
- * **The English Strategy:** Nepal has excellent English media (The Kathmandu Post, Republica, Nepali Times, OnlineKhabar English, Himalayan Times). By using English, you can use powerful, off-the-shelf NLP models (like HuggingFace, SpaCy, or OpenAI/Gemini APIs) right out of the box with 95%+ accuracy.
- * **The "Secret" Workaround for Later:** If you finish the English version early and want to add Nepali sites (like Kantipur or Setopati), you don't need to do Nepali NLP. You can scrape the Nepali text, run it through a translation API (like Google Translate) to English *in the background*, run your English NLP analysis on the translated text, but display the original Nepali text to the user!

2. Is it too hard to analyze these many parameters?

If you try to train a traditional Machine Learning model for each parameter (one model for Sentiment, one for Establishment vs. Alternative, one for Geopolitics), **yes**, it will be too hard and

take too long.**

However, there is a modern way to cheat: **Use Large Language Models (LLMs) as your analysis engine.**

Instead of building complex NLP pipelines, you pass the scraped article text to an LLM API (like OpenAI, Gemini, or a locally hosted model like Llama-3 if you have a good GPU) with a specific prompt.

You can ask the LLM to return a JSON object evaluating the article on your chosen parameters in one go. For example:

```
```json
{
 "primary_entities": ["KP Oli", "Sher Bahadur Deuba"],
 "entity_sentiment": {
 "KP Oli": "Negative",
 "Sher Bahadur Deuba": "Neutral"
 },
 "theme": "Establishment",
 "geopolitical_angle": "None",
 "favored_party": "Nepali Congress"
}
```
```

How to Structure the Project Scope (The Formula)

For an 8th-semester major project, here is how I recommend narrowing it down so it is impressive but completely doable in 3-4 months:

The Core Focus:

- Entity Sentiment (Article Level):** "Who is this specific article praising or criticizing?" (Driven by AI analysis of the text).
- House Bias (Historical Level):** "Over the last 100 articles, Kathmandu Post has been 60% critical of Party A, while Republica has been 40% critical." (This is just basic math/graphs based on the data you collect over time).

What to drop/deprioritize:

* **Geopolitics:** Only run this check if the article contains keywords like "India", "China", "MCC", "BRI", "Embassy", etc. Otherwise, skip it to save compute time.

Next Steps for Execution

If you agree with this English-first, LLM-powered approach, the next step is to design the architecture. We will need:

- A way to scrape news (e.g., Python + BeautifulSoup via RSS feeds).
- A database to store the raw text (e.g., PostgreSQL or MongoDB).
- The Python analysis script to send text to an LLM and save the JSON results.
- A frontend to display the data.

Does this reduced, English-first scope sound good to you? If so, which part of the tech stack (Scraping, Backend, or Frontend) would you like to plan out first?

Question 4:

no, i don't want to go with the pretrained llm(openai, gemini) approach, i want to do it on my own. can't we go with knowledge transfer models by other parties(not have enough understanding of it right now but im going all in for it will be good inbetween) also im not a total nube i ve good software dev backgroud. also wana let you know i ve mac m3 air with 8gb ram for computation purpose.

Answer:

That is actually a fantastic decision. For an 8th-semester engineering major project, building your own Machine Learning pipeline using **Transfer Learning** (fine-tuning pre-trained open-source models) is much more impressive to examiners than just calling the OpenAI API. It shows real engineering depth.

Since you have a solid software development background, you will pick up the ML integration quickly. However, we need to talk about your hardware: **Mac M3 Air with 8GB RAM**.

The Hardware Reality (M3 8GB RAM)

With 8GB of unified memory (shared between CPU and GPU), you cannot run or train large LLMs (like Llama-3 or Mistral) locally. If you try, your Mac will freeze or crash due to Out of Memory (OOM) errors.

The Solution: You don't need massive 8-Billion parameter text generators anyway! For this project, you need **Encoder Models** (like BERT, RoBERTa, or DeBERTa).

- * These models are tiny (100MB to 500MB).

- * They run blazingly fast on your M3 chip using Apple's Metal Performance Shaders (MPS) in PyTorch.

- * They are actually **better** and **faster** at classification and extraction tasks than large LLMs.

The "Do It Yourself" ML Pipeline (Transfer Learning Approach)

To build this without third-party APIs, your backend will use the `transformers` library from Hugging Face. Here is how we break down your requirements into actual ML tasks:

1. Named Entity Recognition (NER)

Goal: Extract the names of politicians or political parties from the article.

- * **How:** You download a pre-trained NER model (e.g., `dslim/bert-base-NER`). When you pass an article text to it, it outputs the exact words that are People (PER) or Organizations (ORG).

- * **Example output:** "Prime Minister [KP Sharma Oli] (PER) addressed the [UML] (ORG) cadre..."

2. Event Clustering (Grouping the News)

Goal: If Republica and Kathmandu Post both publish an article about the "MCC Agreement", how does your system know they are the same story so it can show them side-by-side?

How: You use **Sentence Transformers** (specifically, a tiny model like `all-MiniLM-L6-v2`). This model converts article headlines/summaries into mathematical vectors (embeddings). You calculate the cosine similarity between the vectors. If they are 85% similar, you cluster them together as the "Same Story".

3. Targeted Sentiment Analysis (The trickiest but coolest part)

Goal: Standard sentiment analysis says "This whole article is negative." We don't want that. We want **Aspect-Based Sentiment Analysis (ABSA)**. We want the model to say: "This article is **Negative** towards KP Oli, but **Positive** towards Balen Shah."

How: Transfer Learning! You take an open-source model like `yangheng/deberta-v3-base-absa-v1.1`. You pass it the text AND the entity (which we found in Step 1). It reads the text focusing only on that entity and outputs Positive/Negative/Neutral.

4. The Analytics Engine (House Bias & Establishment vs Alternative)

Once your ML models have processed a database of 1,000 articles, this part is just pure Software Engineering and SQL.

* You write queries to group the data: `SELECT sentiment, count(*) FROM articles WHERE entity = 'RSP' GROUP BY publisher_name;`

* You map certain parties to "Establishment" (NC, UML, Maoist) and others to "Alternative" (RSP, Independents) in your database.

Suggested Tech Stack for your M3 Air

- Backend / API:** `FastAPI` (Python). It is modern, incredibly fast, and pairs perfectly with Machine Learning async tasks.
- Scraping:** `Feedparser` (for RSS feeds - Kathmandu Post, Republica, etc., all have RSS) and `BeautifulSoup4` for full text.
- Machine Learning:** `PyTorch` (configured to use the `mps` device for your M3 chip) and Hugging Face `transformers` pipeline.
- Database:** `PostgreSQL` (Relational is best here to map Articles -> Entities -> Publishers).

Where to start?

Since you are experienced in dev but relatively new to this kind of NLP, I highly recommend we start by proving the ML works on your laptop.

Would you like me to write a small, self-contained Python script using Hugging Face transformers that you can run on your Mac right now? We can do a quick test on **NER (Extracting Nepali Politicians)** or **Sentence Clustering (Grouping two similar headlines)** to see how it performs on 8GB RAM.

Question 5:

BTW,

i ve been learning ML for sometime now and ve some knowledge in it though.

create a detail or outline that i need to learn or to master before diving deep into this project.

Answer:

Since you already have a software development background and some ML knowledge, you can skip the absolute basics (like Linear Regression or basic Pandas) and jump straight into the specialized NLP (Natural Language Processing) and MLOps concepts required for this project.

Given your hardware (M3 Air, 8GB RAM) and your goal of building an Entity-Based Sentiment & Clustering engine, here is a detailed, phase-by-phase learning outline to master before you start coding the main project.

Phase 1: Modern NLP Foundations (The Transformer Architecture)

Since you are not using traditional APIs, you need to understand how modern open-source text models work. Because of your 8GB RAM limit, you should focus entirely on **Encoder-only models** (like BERT, RoBERTa, DeBERTa), which are small (100MB-500MB) but extremely powerful for text analysis.

* **Concepts to Master:***

- * The Attention Mechanism (high-level understanding).
- * Word Embeddings vs. Contextual Embeddings.
- * Encoder vs. Decoder models (Why BERT is better for classification/extraction than LLaMA/GPT).

- * Tokenization (Byte-Pair Encoding, WordPiece) and how it handles out-of-vocabulary words (important for Nepali names).

* **Libraries to Learn:***

- * `transformers` by Hugging Face (The absolute industry standard).
- * `PyTorch` (Focus strictly on creating datasets, dataloaders, and moving tensors to your Mac's GPU).

* **Mini-Goal:** Load a pre-trained `bert-base-cased` model locally on your Mac and pass a sentence through it to view the raw tensor outputs.

Phase 2: Vector Embeddings & Event Clustering (The "Same Story" Engine)

To group articles from *Kathmandu Post* and *OnlineKhabar* that talk about the exact same political event, you need to convert text into mathematics.

* **Concepts to Master:***

- * Sentence Embeddings (converting a whole headline/paragraph into a fixed-size dense vector).
- * Cosine Similarity & Distance Metrics (mathematically measuring how similar two text vectors are).
- * Clustering Algorithms for text: **DBSCAN** or **Agglomerative Clustering** (K-Means is bad for news because you don't know the number of news events 'K' in advance).

* **Libraries to Learn:***

- * `sentence-transformers` (The best library for creating sentence embeddings efficiently).

- * `scikit-learn` (for DBSCAN/Agglomerative clustering).
- * **Mini-Goal:** Scrape 20 headlines, convert them to embeddings using `all-MiniLM-L6-v2` (a tiny, fast model), and write a script that groups the ones talking about the same event.

Phase 3: Information Extraction (Named Entity Recognition - NER)

You need to automatically extract who or what the article is about (e.g., KP Oli, Maoist Center, MCC).

- * **Concepts to Master:**
 - * Sequence Labeling (Token Classification).
 - * Inside-Outside-Beginning (IOB) tagging format.
- * **Libraries to Learn:**
 - * `spaCy` (Great for fast, out-of-the-box NER).
 - * `transformers` pipeline for NER (using models like `dslim/bert-base-NER`).
- * **Mini-Goal:** Pass a Nepali political news article (in English) through an NER model and extract a clean list of all `PERSON` and `ORGANIZATION` entities.

Phase 4: Aspect-Based Sentiment Analysis (ABSA)

Standard Sentiment Analysis classifies an entire text (e.g., "This movie review is positive"). For news, you need ABSA, which determines the sentiment *towards a specific entity* within the text (e.g., "The article is positive about Party A, but negative about Party B").

- * **Concepts to Master:**
 - * Cross-Encoder models vs. Bi-Encoder models.
 - * How to format inputs for ABSA (e.g., feeding the model `[CLS] Text [SEP] Entity [SEP]`).
- * **Libraries to Learn:**
 - * `transformers` (using specific ABSA models like `yangheng/deberta-v3-base-absa-v1.1`).
- * **Mini-Goal:** Write a script that takes the sentence "While Balen Shah's road expansion was largely supported, the traditional parties criticized his methods." and outputs Positive for `Balen Shah` and Negative for `traditional parties`.

Phase 5: Transfer Learning & Fine-Tuning

Out-of-the-box models are trained on Western data. They might not perfectly recognize terms like "Singha Durbar" as an organization/entity, or understand the specific negative context of a word like "syndicate" in Nepali politics. You will likely need to fine-tune a model slightly.

- * **Concepts to Master:**
 - * Transfer Learning (taking a pre-trained model and updating its weights on a small custom dataset).
 - * Creating a custom dataset in Hugging Face `Dataset` format.
 - * The Training Loop (Loss functions, AdamW optimizer, Learning Rate scheduling).
- * **Mini-Goal:** Hand-label 50-100 Nepali political headlines for sentiment, and fine-tune a tiny model (like `distilbert`) on your M3 Mac to predict Nepali-specific political sentiment better.

Phase 6: MLOps and Hardware Optimization (For Mac M3 8GB)

Since you are constrained by 8GB of unified RAM, memory management is critical. If your scraping backend and ML models compete for RAM, your system will crash.

- * **Concepts to Master:**

- * **MPS (Metal Performance Shaders):** Apple's equivalent to NVIDIA's CUDA. You must learn how to move PyTorch models to `device = "mps"` instead of `"cpu"` or `"cuda"`.
- * Batch processing vs. Streaming (processing articles 1 by 1 rather than loading 1000 into RAM at once).
- * Model Quantization (basics of 8-bit precision, though 32-bit encoder models usually fit in 8GB fine).
- * **Libraries to Learn:**
 - * `FastAPI` (To wrap your PyTorch models into a lightweight API).
- * **Mini-Goal:** Set up a FastAPI endpoint that loads an NER model into MPS memory on startup, and accepts POST requests with text, returning extracted entities without crashing your RAM.

Suggested Learning Plan

1. Spend **Week 1** on Phases 1 & 2 (Transformers basics and Sentence Transformers/Clustering). This is the core of the "Ground News" feature.
2. Spend **Week 2** on Phases 3 & 4 (NER and ABSA). This is the analytical engine.
3. Spend **Week 3** on PyTorch MPS optimization and creating your Custom Dataset (Phase 5 & 6).

How comfortable are you with PyTorch and Hugging Face `transformers` currently? We can explore one of these concepts conceptually if you'd like.

Question 6:

also i want to talk about as i already said the project is my 8th sem mejor project so i ve to demo it live infront of inspectors what about that?
again m3 would be problem as can't train it in cloud like google colab or something similar of free tier?

Answer:

This is a very smart realization. Hardware constraints during a live university defense are one of the biggest reasons projects fail on demo day.

Here is the good news: **You do not need to train the model on your Mac, and your M3 Air is actually perfect for the live demo.**

In Machine Learning, there is a massive difference between **Training** and **Inference** (running the model to get a prediction).

1. Training (Do this in Google Colab)

Yes, absolutely use Google Colab! The free tier gives you access to a Tesla T4 GPU with 16GB of VRAM.

- * **The Workflow:** You will write your training scripts, load your custom Nepali news dataset, and fine-tune your Hugging Face models entirely in the cloud on Colab.

- * **The Export:** Once the model finishes training, you simply save the model weights (it will be a folder with some JSON files and a `.safetensors` file, usually around 200MB to 400MB). You download

this folder to your Mac.

2. Live Demo / Inference (Do this on your Mac M3)

For the live demo, your Mac is not *training* anything; it is only doing *Inference* (loading the pre-trained 400MB model into RAM and asking it to classify text).

* *Memory Usage:* A standard NLP model like `RoBERTa` or `DeBERTa` takes up about 500MB of RAM during inference. Even with your 8GB RAM limit, if you close heavy background apps, your FastAPI backend will easily run 2 or 3 of these models simultaneously without crashing.

* *Speed:* Apple's M3 chip is incredibly fast at inference. Using PyTorch's `mps` (Metal) backend, classifying an article will take less than 1 second.

The Foolproof Live Demo Strategy (For Inspectors)

When presenting to inspectors, you want a mix of *guaranteed safety* and *"wow" factor*. Here is how you should structure the defense:

Part 1: The Pre-computed Dashboard (The Safety Net)

Do not rely purely on live internet processing. Have a database (PostgreSQL/MongoDB) already filled with 1,000+ scraped and analyzed articles from the last 7 days.

* When you open the app, it instantly shows the dashboard, the bias charts, and the clustered stories. This proves the system works at scale and gives you beautiful graphs to talk about immediately.

Part 2: The "Live" Test (The Wow Factor)

Create a specific page in your app called "Live Analyzer" just for the demo.

* Have the inspector point to a brand new, breaking news article on *The Kathmandu Post* or *OnlineKhabar (English)*.

* Copy and paste the URL into your app.

* Your Python backend will:

1. Scrape the article text live (takes 1 second).
2. Pass it through your downloaded Colab model on your M3 chip (takes 1-2 seconds).
3. Output the Entities and Sentiment live on screen.

Summary of your Hardware Architecture:

1. *Google Colab (Free):* Data pre-processing, Model Fine-tuning, Model Exporting.
2. *Mac M3 Air:* Web Scraping, Database Hosting, Running the FastAPI backend, Running Model Inference (using the downloaded weights), and Hosting the React/Next.js frontend.

This structure is highly professional and exactly how real machine learning engineering teams deploy local edge models. How does this deployment plan sound for your defense?

Question 7:

also i want to talk about as i already said the project is my 8th sem meyor project so i ve to demo it live infront of inspectors what about that i mean how we will live demo it, live article or media house

categorazitaion or what?

again m3 wouldbn't be problem as can't we train the heavy models in cloud like google colab or something similar of free tier?

Question 8:

ok, these seems nice. we will further refine it.

for now i want to present this idea to my team-mates. can you summurize it or make it presentable?

Answer:

Here is a structured, presentation-ready project pitch you can share with your teammates. You can copy and paste this into a document, Slack/Discord, or use it as an outline for a slideshow.

Project Pitch: "Ground News" for Nepal 🇳🇵

****An AI-Powered Media Bias and News Aggregation Platform****

1. The Problem

In Nepal, media bias isn't a simple "Left vs. Right" ideological divide like in the West. Politics here is driven by personalities, alliances, and the divide between traditional power (Establishment) and new forces (Alternative). Currently, there is no tool for readers to see how different media houses frame the exact same event or which politicians they subtly favor.

2. The Solution

We are going to build an automated news aggregator that scrapes major news portals, groups articles about the same event together, and uses Machine Learning to analyze the sentiment/bias of the reporting.

****To ensure we finish this in 3-4 months, our MVP (Minimum Viable Product) will focus exclusively on English-language Nepali media**** (e.g., Kathmandu Post, Republica, OnlineKhabar English). This allows us to use highly accurate, modern NLP models without getting stuck in the complexity of under-resourced Nepali NLP.

3. Core Features (What the User Sees)

- * ****Event Clustering (Headline Stacking):**** If three different news sites report on a major scandal, our app groups those three articles side-by-side so the user can instantly compare the headlines.
- * ****Aspect-Based Sentiment Analysis (ABSA):**** Instead of saying "This article is negative," our AI reads the text and says: **"This article is critical of KP Oli, but neutral towards Balen Shah."**
- * ****Media House Bias Tracking:**** Over time, the app builds a dashboard graph showing historical leanings (e.g., "Over the last 100 articles, Publisher X has historically favored the Establishment").

4. The Technical Strategy (How We Build It)

We are avoiding expensive API calls (like OpenAI) by building our own ML pipeline using ****Transfer Learning**** (fine-tuning open-source models).

- * ****Training (The Cloud):**** We will fine-tune small, highly efficient transformer models (like BERT or DeBERTa) on free Google Colab GPUs using custom Nepali political datasets we create.

* **Inference (Local):** These trained, lightweight models will be downloaded and run entirely locally on our machines (optimized for Apple Silicon / MPS) for blazingly fast, free text analysis.

5. Our Tech Stack

- * **Data Ingestion:** Python (`BeautifulSoup`/`Feedparser`) running as automated scheduled jobs to scrape RSS feeds.
- * **Database:** PostgreSQL (to store articles, publishers, and extracted entities).
- * **Machine Learning Backend:** FastAPI (Python) running Hugging Face `transformers`:
 - * **Sentence-Transformers:** To mathematically group similar articles together.
 - * **NER (Named Entity Recognition):** To pinpoint politicians' names in the text.
 - * **ABSA (Aspect-Based Sentiment):** To judge the framing of those specific names.
- * **Frontend End:** React or Next.js (for a clean, modern dashboard).

6. The "Wow Factor" for the Final Defense

We won't just present a static website. Our demo will have two parts:

1. **The Live Dashboard:** Showing rich graphs and clustered news generated automatically over the previous month.
2. **The Developer Playground:** We will let the inspectors paste *any* paragraph of political news into our app live. Our locally hosted ML pipeline will analyze the text, extract the politicians' names, and judge the sentiment in under a second.

Discussion Points for the Team:

- * Who wants to handle the **Data Scraping & Database** (Python/SQL)?
- * Who wants to handle the **Machine Learning Pipeline & API** (Hugging Face/FastAPI)?
- * Who wants to handle the **Frontend Dashboard & Data Visualization** (React/Next.js)?

Question 9:

these are technical points. i will present this but later. for now i just want to present the core idea.

Answer:

Congratulations on finalizing the project! For an 8th-semester major project proposal, you need a structured document that sounds academic, technically rigorous, and well-planned.

Here is a comprehensive framework for your project proposal. You can plug this directly into your university's official proposal template.

Project Title Ideas

1. **NewsLens Nepal:** An AI-Powered News Aggregation and Aspect-Based Bias Analysis System
2. **MediaPulse:** Automated Event Clustering and Entity-Based Sentiment Analysis in Nepali Journalism
3. **Echoes & Angles:** Profiling Media Bias in Nepal using Transfer Learning and NLP

1. Introduction

The consumption of digital news in Nepal has grown exponentially, bringing with it the challenge of media bias and sensationalism. Unlike Western politics, which is cleanly divided into ideological spectrums (Left vs. Right), Nepali media bias is heavily driven by personalities, political factions, and the dynamic between "Establishment" (traditional power) and "Alternative" (new political forces).

This project aims to build an automated news aggregation platform focusing on English-language Nepali media. By leveraging Natural Language Processing (NLP) and Machine Learning, the system will cluster articles covering the same event side-by-side and perform Aspect-Based Sentiment Analysis (ABSA) to mathematically reveal how different media houses frame specific political entities.

2. Problem Statement

1. ****Fragmented Reporting:**** Readers lack a centralized platform to directly compare how different news publishers report on the exact same event.
2. ****Hidden Biases:**** Traditional sentiment analysis is insufficient for news. A generic "negative" classification fails to identify *who* the article is attacking or defending.
3. ****Lack of Accountability:**** There is no data-driven historical tracking of media house bias toward specific political figures or parties in the Nepali context.

3. Project Objectives

- * ****Automated Data Ingestion:**** To develop an automated scraping engine that collects daily articles from major English-language Nepali news portals (e.g., Kathmandu Post, Republica, OnlineKhabar).
- * ****Semantic Event Clustering:**** To implement transformer-based vector embeddings to group distinct articles that discuss the same underlying news event.
- * ****Entity Extraction & ABSA:**** To fine-tune and deploy NLP models for Named Entity Recognition (NER) to extract political figures, and Aspect-Based Sentiment Analysis (ABSA) to determine the media's sentiment specifically directed at those entities.
- * ****Data Visualization:**** To build a web dashboard that visualizes daily clustered news and historical media house bias "report cards."

4. System Architecture & Component Interactions

(You will draw this as a Block Diagram later, but here is the written explanation)

The system operates in a multi-stage pipeline:

1. ****Data Collection Module (Cron/Scraper):**** Python scripts periodically parse RSS feeds and scrape the full text of new articles. The raw text is cleaned and stored in the database.
2. ****ML Processing Module (The AI Engine):****
 - * ***Step A (Vectorization):*** The article headline/summary is converted into a mathematical vector using a lightweight Sentence Transformer.
 - * ***Step B (Clustering):*** The system compares this vector to recent articles using Cosine Similarity. If the similarity is above a threshold (e.g., 85%), it groups them into an "Event Cluster".
 - * ***Step C (NER):*** An NLP model scans the text and extracts key entities (e.g., "Maoist Center", "KP Oli").

* **Step D (Sentiment):** An ABSA model evaluates the text surrounding the extracted entities and assigns a polarity score (Positive, Negative, Neutral).

3. **Backend API (FastAPI):** Serves as the bridge between the Database, the ML pipeline, and the user interface. It handles search queries, data aggregations, and live ML inference requests.

4. **Frontend Dashboard (React/Next.js):** Consumes the API and renders the interactive user interface, including the stacked headlines and bias charts.

5. Technology Stack

The project adopts a "Train in the Cloud, Deploy at the Edge" MLOps strategy to maximize the efficiency of local Apple Silicon hardware.

* **Machine Learning & NLP:**

* **Frameworks:** PyTorch, Hugging Face `transformers`, `sentence-transformers`.

* **Training Infrastructure:** Google Colab (GPU) for fine-tuning pre-trained models (e.g., BERT, DeBERTa) on custom Nepali political context.

* **Inference Infrastructure:** Apple Metal Performance Shaders (MPS), explicitly optimizing PyTorch to run text inference fast and locally on the Mac M3's unified memory.

* **Backend & APIs:** Python 3.10+, FastAPI (for high-performance asynchronous API endpoints), SQLAlchemy (ORM).

* **Data Collection:** `BeautifulSoup4`, `Feedparser`, `Celery` (for task scheduling).

* **Database:** PostgreSQL (to handle relational data between Articles, Entities, and Publishers).

* **Frontend:** React (Next.js framework) with Tailwind CSS and Recharts/Chart.js for data visualization.

6. Scope and Limitations

* **Language Limitation:** To maintain high NLP accuracy and complete the project within the academic timeframe, the system will exclusively process English-language news portals.

* **Entity Focus:** The sentiment analysis will focus strictly on political and geo-political entities (parties, politicians, nations), ignoring general topics (sports, entertainment).

7. Expected Deliverables

1. A fully functional web application exhibiting live event clustering and bias analysis.

2. An interactive "Developer Playground" mode demonstrating localized NLP inference running in real-time.

3. A comprehensive database capturing at least one month of media bias trends across major Nepali news outlets.

Next Steps for You:

When you are ready, I can help you draft the visual diagrams (like Data Flow Diagrams, Entity-Relationship Models for your Database, or System Architecture diagrams).

Does this structure give you everything you need for the written proposal?

Question 10:

yes, this is nice will help me while writing the paper wouldn't be enough but don't you bother i'll handle. i want to go deep into architecture, working and teach coz now i myself isn't sure about it and i don't want to over-promise and later under deliver. having said that im will and will learn everything it requires. but now first i have to create the proposal.

Answer:

It's very wise to avoid over-promising in your proposal. A strong engineering project isn't about claiming you will achieve 100% accuracy; it's about showing a well-thought-out architecture and understanding the trade-offs of the technologies you choose.

Let's break down the deeper architecture, working mechanisms, and the actual technologies you will use, so you understand **exactly** how this works and what you are committing to.

Deeper Dive: System Architecture & Workflow

We are building a pipeline. Think of it as a factory assembly line where raw data comes in one end, goes through various shaping processes, and comes out as organized, visualized insights at the other.

1. The Scraping & Ingestion Layer

* **The Reality:** Scraping HTML directly is fragile; if a site changes its layout, your scraper breaks.
* **The Safe Promise:** You will use **RSS Feeds** as the primary source. Almost all major news sites (Kathmandu Post, Republica, OnlineKhabar English) provide RSS feeds.

* **How it works:**

1. A Python script (using the `feedparser` library) runs every X hours.
2. It checks the RSS feed for new URLs.
3. It downloads the HTML of the new URLs and uses `BeautifulSoup` to extract the main article body.
4. It saves the raw text, title, publication date, and source to your PostgreSQL database.

2. The Vectorization & Clustering Layer (Grouping the News)

* **The Reality:** Traditional text matching (checking if words match) fails because "KP Oli criticized" and "Former Prime Minister lashed out" mean the same thing but use different words.

* **The Safe Promise:** You will use **Semantic Search via Dense Embeddings**.

* **How it works:**

1. A new article comes in. You send its Title + First Paragraph to a lightweight model (e.g., `sentence-transformers/all-MiniLM-L6-v2`).
2. The model outputs an "embedding"—a list of 384 numbers that represents the **meaning** of the text.
3. You compare this list of numbers to the numbers of articles from the last 48 hours using **Cosine Similarity**.
4. If the distance between the vectors is small (threshold > 0.85), your database links them together under the same `Event\_ID`.

3. The Extraction Layer (Named Entity Recognition - NER)

* **The Reality:** Pure standard NER will find **all** people and organizations, including completely

irrelevant ones (like finding the name of a journalist or a random company).

* **The Safe Promise:** You will extract entities but filter them against a predefined "Target Dictionary" (e.g., a list of major Nepali politicians and parties you care about tracking).

* **How it works:**

1. Pass the article text through a pre-trained NER model (e.g., `dslim/bert-base-NER` via the HuggingFace `pipeline`).
2. The model outputs a list of names.
3. Cross-reference this list with your dictionary to keep only the relevant political actors. Save these to the database linked to the article.

4. The Sentiment Layer (Aspect-Based Sentiment Analysis - ABSA)

* **The Reality:** This is the hardest part. You cannot simply use textblob or basic sentiment because an article might say "The brilliant Balen Shah arrested the corrupt official." Standard sentiment says "Mixed". We need to know it's positive for Balen, negative for the official.

* **The Safe Promise:** You will utilize a pre-trained ABSA model and focus on predicting the sentiment direction (Positive/Negative/Neutral) specifically toward the entities extracted in step 3.

* **How it works:**

1. You take the extracted entity (e.g., "Balen Shah") and the sentence(s) he appears in.
2. You format the input for a model like `yangheng/deberta-v3-base-absa-v1.1`. Input format usually looks like: `[CLS] The brilliant Balen Shah arrested the corrupt official. [SEP] Balen Shah [SEP]`.
3. The model outputs the probability of Positive, Negative, or Neutral toward that specific entity. Save this score to the database.

The Tech Stack (What you are actually installing)

To be confident when writing your proposal or talking to your professors, this is the exact stack you will need to learn and use.

Backend / API:

* **Python 3.10+** (The core language for the entire backend).

* **FastAPI:** Why? Because it handles asynchronous requests brilliantly, which is necessary when waiting for external scrapers or ML models to finish processing. It's much faster than Django or Flask.

* **Uvicorn:** The ASGI server to run FastAPI.

* **SQLAlchemy ORM:** How you interact with your database using Python code instead of raw SQL queries.

Machine Learning (The Core):

* **PyTorch (`torch`):** The engine moving the math. Crucially, you will write initialization code that checks `torch.backends.mps.is\_available()` to force PyTorch to use your Mac's M3 GPU instead of the CPU.

* **HuggingFace `transformers`:** The library to download and run the NER and ABSA models.

* **`sentence-transformers`:** The specific library optimized for creating embeddings for clustering.

* **(Optional but recommended) Google Colab:** Only used if you decide you need to fine-tune a model on a custom Nepali dataset. You do not need this just to run pre-trained models.

****Database:****

- * ****PostgreSQL:**** Handles the relational mappings perfectly (e.g., An `Event` has many `Articles`. An `Article` has many `Entities`. An `Entity` tied to an `Article` has a `Sentiment\_Score`).
- * ****pgvector (Optional Extension):**** If you want to get very advanced, this PostgreSQL extension allows you to store the 384-number vector embeddings directly in the database and run similarity queries inside SQL.

****Frontend:****

- * ****Next.js (React):**** Standard, robust, and handles routing and API fetching smoothly.
- * ****Tailwind CSS:**** For fast, custom styling.
- * ****Recharts or Chart.js:**** For rendering the historical bias charts and sentiment graphs.

What to write in the proposal vs. What to hide

- * ****Write about:**** The pipeline (Scraping -> Vectorization -> NER -> ABSA -> Visualization). Name the specific technologies (FastAPI, PyTorch, HuggingFace, React). State that you are optimizing for local Apple Silicon hardware.
- * ****Do not promise:**** 100% sentiment accuracy. Perfect Nepali language support. Real-time sub-second scraping (news takes time to publish anyway).

Does this breakdown help solidify your understanding of how the pieces fit together? If you are comfortable with this, the next step is usually designing the database schema (what tables you need), which makes the whole project crystal clear.

Question 11:

don't we have to finetune or train also i want to mention i have worked with scrapy for scraping in the past.

Answer:

Answer:

I have generated a `PROJECT\_CONTEXT.md` file in the root of your workspace.

This file distills our entire conversation into a clean, professional markdown document. It covers the problem statement, the core features, the pipeline architecture, the exact machine learning strategy (Transfer Learning), the tech stack, and the live demo strategy.

You can share this `.md` file directly with your teammates, or they can copy and paste the text into whatever AI tools they use (ChatGPT, Claude, etc.) with a prompt like: **"Here is the context for my 8th-semester engineering project. Keep this in mind for all our future conversations."**

This will immediately bring them (and their AI assistants) up to speed on exactly what we are building and how it works as well-scoped, professional architecture looks like for this project. Ready for the next steps?

Question 12:

where's the file i don't see it?

Answer:

Answer:

Answer:

Answer:

I have just triggered a command to open the `PROJECT\_CONTEXT.md` file directly in your VS Code editor. It should pop up as an active tab for you now.

The issue might be related to the fact that your workspace folder name (`8th sem project `) has a trailing space at the very end, which sometimes causes display glitches in the VS Code file explorer.

If it's still not opening or visible, here is the exact content I placed in it so you can quickly copy it:

```markdown

# 🇳🇵 Project Context: "Ground News" Nepal

**\*\*Document Purpose:\*\*** This document serves as the master context file for our 8th-Semester Major Project. Share this with the team and feed it to any AI assistants you use to ensure everyone is fully synced on the vision, architecture, and technology stack.

---

### ## 1. Vision & Problem Statement

In Nepal, media bias isn't a simple "Left vs. Right" ideological divide. It is driven by personalities, political alliances, and the friction between the "Establishment" (traditional power players like NC,

UML, Maoist) and "Alternative" forces (RSP, independent mayors like Balen).

Currently, readers have no centralized way to compare how different media houses frame the exact same event. Our project solves this by building an **AI-powered news aggregator** that automatically groups identical stories from different publishers and analyzes the sentiment/bias toward specific political entities.

*\*To ensure project completion within 3-4 months, we are exclusively processing English-language Nepali news portals (e.g., Kathmandu Post, Republica, OnlineKhabar English).\**

---

## ## 2. Core Features (MVP)

1. **Headline Stacking (Event Clustering):** Grouping articles from different publishers that cover the exact same event so users can instantly spot framing differences.
2. **Entity-Based Bias Scanner (ABSA):** Using AI to read an article and say, *"This article is highly critical of KP Oli, but neutral towards the RSP"* rather than just labeling the whole article as "negative."
3. **Media House Report Cards:** A dashboard showing the historical bias of news portals (e.g., showing if a publisher consistently writes negative articles about a specific party over a 30-day period).

---

## ## 3. System Architecture & Pipeline

The system operates as a continuous data pipeline:

1. **Ingestion Layer:** **Scrapy** spiders run on a schedule, pull new articles via RSS feeds/HTML, and store the raw text and metadata in a database.
2. **Vectorization & Clustering Layer:** The incoming article is converted into a mathematical vector using `sentence-transformers`. It is compared to recent articles using Cosine Similarity. If the similarity score is high (e.g.,  $>0.85$ ), they are linked together as the same "Event".
3. **Extraction Layer (NER):** A Named Entity Recognition model scans the text to extract relevant people and organizations (filtered against a custom dictionary of Nepali political actors).
4. **Sentiment Layer (ABSA):** An Aspect-Based Sentiment Analysis model evaluates the context surrounding the extracted entities, outputting a Polarity Score (Positive, Negative, Neutral).
5. **Serving Layer:** A **FastAPI** backend serves this structured data to the frontend dashboard.

---

## ## 4. Machine Learning & Hardware Strategy

To ensure professional-grade ML without massive costs, we follow a **"Train in the Cloud, Deploy at the Edge"** MLOps strategy.

*\*No "Training from Scratch":* We will perform **Transfer Learning / Fine-Tuning**.

*\*The Process:* We will take pre-trained lightweight models (e.g., `distilbert`, `deberta-v3-base`) and fine-tune them on a custom, hand-labeled dataset of ~500-1000 Nepali political sentences using free GPUs on **Google Colab**.

**Local Inference:** The fine-tuned weights (~300MB) will be deployed locally inside our FastAPI app. To accommodate 8GB RAM limitations (e.g., Mac M3 Air), PyTorch will be optimized to use Apple's Metal Performance Shaders (MPS), allowing instant, free local inference without crashing.

---

## ## 5. Technology Stack

**Data Scraping:** Python, Scrapy (handles async requests and pipelines).  
**Database:** PostgreSQL (Ideal for relational data: Events -> Articles -> Entities -> Sentiment).  
**Machine Learning:** PyTorch, Hugging Face `transformers`, `sentence-transformers`.  
**Backend API:** FastAPI (Async, Python 3.10+), SQLAlchemy ORM.  
**Frontend:** React / Next.js, Tailwind CSS, Recharts/Chart.js for bias visualization.

---

## ## 6. Live Demo Strategy (For Inspectors)

To avoid the risks of live-scraping failing during the presentation, our demo will consist of two parts:

- The Pre-Baked Dashboard:** Showcasing weeks of historic, clustered, and analyzed data to prove the pipeline ran successfully over time.
- The Developer Playground:** A dedicated UI tab where the inspector can paste *any* paragraph of political news. Our local FastAPI backend will run the text through our fine-tuned NLP models live, extracting the entities and calculating sentiment in under a second.

---

-----

Question 13:

give me the expected deliverable or proposed output of this project.

Answer:

Here is a structured list of the "Expected Deliverables" (Proposed Outcomes) that you can copy directly into your project proposal. It breaks down the abstract vision into concrete, gradable software components that the inspectors will evaluate.

---

### ### Expected Deliverables

Upon completion of this 8th-semester major project, the following core components and artifacts will be delivered:

#### **1. Automated Data Ingestion Subsystem**

- A robust, scheduled web scraping pipeline built with Python and Scrapy.
- The system will successfully parse RSS feeds and extract full-text articles from at least three major English-language Nepali news portals (e.g., The Kathmandu Post, Republica, OnlineKhabar English).
- A populated PostgreSQL database containing a rich dataset of historical news articles stored with their relevant metadata (source, timestamp, raw text).

## **\*\*2. Custom Machine Learning Pipeline (Inference Engine)\*\***

- \* **\*\*Semantic Clustered Data:\*\*** A functional implementation of vector embeddings (via `sentence-transformers`) capable of algorithmically grouping similar news articles into unified "Event Clusters" with a high degree of cosine similarity.
- \* **\*\*Fine-tuned NLP Models:\*\*** Deployment of locally hosted, lightweight transformer models (e.g., DistilBERT/DeBERTa) running on Apple MPS. This includes:
  - \* A custom Named Entity Recognition (NER) pipeline capable of reliably isolating relevant Nepali political entities from the text.
  - \* An Aspect-Based Sentiment Analysis (ABSA) module that outputs distinct polarity scores (Positive, Negative, Neutral) aimed specifically at the extracted entities.

## **\*\*3. Backend Application Programming Interface (API)\*\***

- \* A high-performance, asynchronous REST API developed using FastAPI.
- \* Endpoints designed to serve clustered news data, historical bias metrics, and handle live ML inference requests dynamically.

## **\*\*4. Interactive Frontend Web Dashboard (The MVP)\*\***

- \* A responsive, user-facing web application built with React/Next.js and styled with Tailwind CSS.
- \* **\*\*Core Views include:\*\***
  - \* **\*\*The Aggregator View:\*\*** A dynamic newsfeed where identical events from different publishers are stacked side-by-side for immediate comparison.
  - \* **\*\*The House Bias Dashboard:\*\*** Data visualization charts outlining the historical sentiment footprint of specific news publishers toward various political parties.
  - \* **\*\*The Developer Playground:\*\*** A dedicated testing interface allowing users/instructors to paste raw text and view the local ML pipeline process the Named Entities and Sentiment live in under a second.

## **\*\*5. Comprehensive Documentation\*\***

- \* The final academic project report detailing the system architecture, MLOps fine-tuning methodology, and limitations.
- \* A structured GitHub repository containing clean, documented source code, database schemas, and README instructions for environment setup.

-----

Question 14:

im still confuse on what parameters we will run or system.

Answer:

It is completely normal to feel confused at this stage. When we talk about "bias," it sounds like an abstract concept. But to build a system, we have to turn "bias" into strict database columns (parameters) that our code can actually measure and store.

Let's demystify this. Here are the exact **\*\*four parameters\*\*** your system will calculate and store for every piece of news.

---

### ### 1. The "What" Parameter: Event Cluster ID

Whenever an article is scraped, the system checks if it belongs to a story we already know about.

- \* **What we measure:** Cosine Similarity Score (0 to 1).
- \* **How it works:** If *Republica* publishes an article about a budget speech, and its text is 88% (0.88) similar to a *Kathmandu Post* article from yesterday, they both get assigned the same ``Event_ID: 1045``.
- \* **Result:** This allows your frontend to stack them side-by-side.

### ### 2. The "Who" Parameter: Extracted Entities

Your system will not just scan blindly; it will look for specific actors.

- \* **What we measure:** A list of ``PERSON`` or ``ORGANIZATION`` tags found in the text.
- \* **How it works:** You will have an internal dictionary. When the ML reads the text, it extracts parameters like:
  - \* ``Entities Found: ["KP Oli", "Nepali Congress", "RSP"]``

### ### 3. The "How" Parameter: Entity Polarity (Sentiment)

This is the core standard for measuring bias in your project. We don't measure if the article is "happy" or "sad." We measure how it treats the people inside it.

- \* **What we measure:** A numerical probability for three states: **Positive, Neutral, or Negative**.
- \* **How it works:** For **every single entity** found in step 2, the ML model outputs a score.
  - \* **Example Output:** ``KP Oli`` -> Negative (0.92 certainty)
  - \* **Example Output:** ``RSP`` -> Neutral (0.80 certainty)

### ### 4. The "Long-Term Bias" Parameter: House Tag

This is calculated historically on your backend rather than by the ML model.

- \* **What we measure:** The ratio of Positive vs. Negative articles a publisher writes about specific groups.
- \* **How it works:** You group entities into two major buckets: **Establishment** (NC, UML, Maoist, old guard) and **Alternative** (RSP, Balen, new guard).
- \* **Result:** Over a month, your system calculates: *Kathmandu Post* has a 65% negative polarity towards Alternative entities, but only 30% negative polarity towards Establishment entities.\*

---

### ### Visualizing the Parameters (What goes in your Database)

To make it concrete, imagine your Scrapy spider just downloaded a new article from *OnlineKhabar*. After passing through your ML pipeline, here is exactly what gets saved as parameters in your database:

```
```json
{
  "article_id": 9923,
  "publisher": "OnlineKhabar English",
  "title": "Ruling coalition heavily criticizes new independent mayors.",
  "event_cluster_id": 402,
```

```

"analysis_parameters": [
  {
    "entity": "Ruling Coalition",
    "entity_type": "Establishment",
    "sentiment": "Positive",
    "confidence_score": 0.88
  },
  {
    "entity": "Independent Mayors",
    "entity_type": "Alternative",
    "sentiment": "Negative",
    "confidence_score": 0.94
  }
]
}
...

```

Summary

When your professors ask, "What exactly are you measuring?", your answer is:

"We are measuring mathematical Event Similarity to group articles, extracting specific Political Entities via NER, and measuring the Aspect-Based Sentiment Polarity directed at those specific entities."

Does seeing it mapped out like a database entry help clear up what the system is actually being measured?

Question 15:

yes, that gave so much clarity but I know it will we only cluster, analyze, store the stories/article on the basis of headline or the whole of article?

Answer:

This is a brilliant engineering question. It is exactly the kind of question a professor will ask during your defense, because making the wrong choice here will cause your servers to crash or your backend to become incredibly slow.

The Short Answer

We will use **Title + The First Paragraph (Summary)** for Clustering, and we will use the **Full Article Body** for Entity Extraction (NER) and Sentiment Analysis (ABSA).

Here is exactly why we split it up this way:

1. For Clustering (Grouping "Same Story"): Use Title + First Paragraph

* **Why NOT the whole article?** Transformer models designed for clustering (like `sentence-transformers`) have a strict "token limit" (usually 512 tokens, which is about 300 words). If you pass an entire 1,500-word article, the model literally chops off everything after the first 300 words anyway. Furthermore, comparing whole articles mathematically takes way too much RAM and compute time.

* **Why Title + 1st Paragraph?** In journalism, the "Inverted Pyramid" rule means the most crucial facts (Who, What, When, Where) are always in the Headline and the First Paragraph. By creating vector embeddings from just this short text, your clustering will be extremely fast, memory-efficient, and highly accurate.

2. For Entity Extraction (NER): Use the Full Article

* **Why?** A politician might not be mentioned in the headline, but they might be quoted or criticized heavily in paragraph 4. If you only scan the headline, you will miss crucial data.

* **How we handle it technically:** Since NER models also have token limits, your backend will split the article into sentences or small chunks (e.g., looping through 5 chunks of 200 words). It passes each chunk to the NER model to ensure it finds every single political entity mentioned anywhere in the text.

3. For Sentiment (ABSA): Use Only the Sentences Containing the Entity

* **Why NOT the whole article?** If you ask a model "What is the sentiment of this whole 1,000-word article towards KP Oli?", the model will get confused by paragraphs talking about the economy or other politicians.

* **How we handle it technically:**

1. In Step 2, our NER found "KP Oli" in Paragraph 3 and Paragraph 6.
2. We extract *only* Paragraph 3 and Paragraph 6.
3. We feed *only those specific paragraphs* into the ABSA (Sentiment) model to evaluate them.
4. If Paragraph 3 is "Negative" and Paragraph 6 is "Negative", we store an overall Negative score for that entity in the database.

Summary of the Data Flow

To make this crystal clear for your proposal or your team, here is the exact data logic:

1. **Scrapy downloads:** Title, First Paragraph, and Full Body Text.
2. **Clustering Engine gets:** `Title + First Paragraph` (Fast, efficient matching).
3. **NER Engine gets:** `Full Body Text` (Analyzed in chunks to find everyone mentioned).
4. **Sentiment Engine gets:** `Specific sentences` (Only the parts where the person was actually named, to measure how they are treated).

By designing the system this way, your M3 Mac (8GB RAM) will be able to process hundreds of articles smoothly without running out of memory. Does this separation of concerns is a hallmark of good software engineering best practice in modern MLOps practices.

Question 16:

ok nice things becoming very clear, but i just wana know or just asking, are we gone use langchain in any part or can do most of it in langchain for ex: Since NER models also have token limits, your backend will split the article into sentences or small chunks (e.g., looping through 5 chunks of 200 words). It passes each chunk to the NER model to ensure it finds every single political entity mentioned anywhere in the text. here we can use it?

Answer:

That is an excellent question! Thinking about using LangChain for chunking and orchestration shows you are up-to-date with modern AI engineering tools.

The Short Answer:

****You can, but you shouldn't.**** While LangChain has tools for chunking text, it is heavily designed for Large Language Models (LLMs like OpenAI or Llama-3) and building RAG (Retrieval-Augmented Generation) pipelines. For our specific use case (using small **Encoder** models like BERT locally), introducing LangChain will add unnecessary overhead and bloat to your project.

Here is a detailed breakdown of why we should avoid LangChain for this specific architecture, and what we will use instead.

Why we should AVOID LangChain for this project:

1. It is Built for Generative Text (LLMs), Not Classification

LangChain's entire ecosystem is built around chaining prompts, hitting APIs (like OpenAI), and generating text (chatbots, summarization).

** Our project doesn't generate text. We are doing ****Classification**** (Sentiment) and ****Extraction**** (NER). LangChain is actually quite bad/clunky at handling strict classification outputs from local HuggingFace ``transformers``.*

2. Too Much Overhead (Bloat)

LangChain has become massive. Importing it brings in hundreds of dependencies. For a FastAPI backend that needs to be fast and memory-efficient to run on your 8GB RAM Mac, importing LangChain just to chunk a text file is like bringing a sledgehammer to kill a mosquito.

3. Hugging Face pipelines already do this perfectly

The tools you need for tokenization, chunking, and routing data to local models are already built directly into the Hugging Face ``transformers`` library, which we are heavily relying on.

How we handle Chunking without LangChain (The "Native" Way)

You mentioned the exact scenario: splitting an article into chunks so the NER model doesn't hit its token limit.

Instead of LangChain's ``RecursiveCharacterTextSplitter``, we will use the native Hugging Face

`Tokenizer`. It is safer and much more accurate because it splits the text based on the *exact tokens* the local model expects, not just character counts.

Here is a conceptual example of how simple and clean this is in pure Python/HuggingFace:

```
```python
from transformers import AutoTokenizer, pipeline

1. Load the tokenizer and the NER pipeline
tokenizer = AutoTokenizer.from_pretrained("dslim/bert-base-NER")
ner_pipeline = pipeline("ner", model="dslim/bert-base-NER", tokenizer=tokenizer)

def extract_entities_from_long_article(full_article_text):
 # 2. Tokenize the long article, automatically chunking it into blocks of 512 tokens
 # returning overlapping chunks so we don't accidentally split a politician's name in half
 tokens = tokenizer(
 full_article_text,
 max_length=512,
 truncation=True,
 stride=50, # 50 token overlap between chunks
 return_overflowing_tokens=True,
 return_offsets_mapping=True
)

 all_entities = []

 # 3. Process each chunk
 for chunk in tokens["input_ids"]:
 # Convert tokens back to text snippet
 text_snippet = tokenizer.decode(chunk, skip_special_tokens=True)
 # Run NER on the snippet
 entities_in_chunk = ner_pipeline(text_snippet)
 all_entities.extend(entities_in_chunk)

 return all_entities
```
```

When SHOULD you use LangChain?

If we decided to scrap the "Train our own local models" idea and instead said, "Let's just pay OpenAI \$0.01 per article and use the GPT-4 API to extract entities and sentiment," then LangChain would be perfect.

But because our primary goal is a custom-trained, locally run inference engine using PyTorch and Hugging Face (which is a much more impressive engineering feat for a university project), native Hugging Face pipelines are the correct tool for the job.

Does that explanation clear up where LangChain fits (and doesn't fit) into modern MLOps architectures?

Question 17:

wow that's hell of a explanation nice, please tell or mentions the tools and technologies we will use and try to keep it feasible.

Answer:

I'm glad that explanation helped! Let's solidify the exact technology stack. We will keep this stack feasible, modern, and perfectly suited for your hardware (Mac M3 Air, 8GB RAM).

This is the definitive list of tools and technologies you should include in your proposal and rely on for the build.

The Final "Ground News Nepal" Tech Stack

1. Data Ingestion & Storage (The Foundation)

- * **Python 3.10+**: The core language connecting all backend components.**
- * **Scrapy**: The industry standard for web scraping. It is asynchronous, handles rate limiting (so you don't get blocked by news sites), and easily extracts data via XPath/CSS selectors.**
- * **PostgreSQL**: The database. It is robust, handles relational data perfectly (linking Articles to Entities and Sentiments), and can easily handle thousands of records generated by daily scraping.**
- * **SQLAlchemy / SQLModel**: The Object-Relational Mapper (ORM) for Python. This allows you to interact with your PostgreSQL database using Python classes instead of writing raw SQL strings in your code.**

2. The Machine Learning Engine (The Brains)

- * **Hugging Face `transformers`**: The library to download, load, and run our local NLP models (for NER and Sentiment).**
- * **`sentence-transformers`**: A specialized library built on top of Hugging Face designed specifically for generating text embeddings (vectors) quickly. We will use this for the event clustering.**
- * **PyTorch (`torch`)**: The underlying math engine. We will explicitly configure it to use the `mps` (Metal Performance Shaders) backend so it utilizes your Mac M3's GPU instead of the slower CPU.**
- * **Scikit-learn**: We will use this briefly for the actual clustering math (e.g., using `cosine\_similarity` to compare the vectors generated by sentence-transformers).**
- * **Google Colab (Free Tier)**: The cloud environment we will use *only* for fine-tuning our models on our custom dataset, saving your Mac from memory crashes during training.**

3. Backend API (The Orchestrator)

- * **FastAPI**: The web framework for building the API. It is incredibly fast, natively asynchronous (which is perfect for waiting on Scrapy or ML predictions), and automatically generates Swagger documentation for your API endpoints.**
- * **Uvicorn**: The lightweight web server that actually runs the FastAPI application.**

4. Frontend Web Dashboard (The UI)

- * **Next.js (React Framework)**: The industry standard for building robust React applications. It allows for server-side rendering (if needed) and clean routing.
- * **Tailwind CSS**: A utility-first CSS framework for rapidly building a clean, modern, and responsive user interface without writing custom CSS files.
- * **Recharts** (or **Chart.js**): A charting library for React to build the "House Bias Dashboard" (e.g., bar charts showing Positive/Negative ratios).

Why this Stack is Highly Feasible:

1. **No APIs to pay for:** By using Hugging Face and local inference, your running cost is exactly \$0 after initial setup.
2. **No RAM crashes:** By splitting the workflow (extracting in chunks, clustering via small vectors, running via FastAPI), your 8GB Mac will handle instructions sequentially without running out of unified memory.
3. **Industry Relevant:** Every single tool here (FastAPI, Next.js, PyTorch, Hugging Face) is currently at the bleeding edge of the software industry. Mastering these will look fantastic on your resume.

With this stack defined, your project proposal should be fully fleshed out! Would you like to review any specific component again, or are you ready to finalize your proposal document?

Question 18:

can please forget about the part of air m3. you don't need to things every aspect from this point only and mention it. and give me it in list.

Answer:

Understood. Removing the hardware constraints, here is the clean, definitive list of technologies you will use for the project.

This is formatted as a direct list, ready to be pasted into your project proposal under the "Technology Stack" section.

Proposed Technology Stack

1. Data Ingestion

- * **Language:** Python 3.10+
- * **Web Scraping Framework:** Scrapy (for asynchronous crawling and data extraction pipelines)
- * **Parsing Parsing:** BeautifulSoup4 / lxml

2. Data Storage

- * **Relational Database:** PostgreSQL (Primary storage for articles, entities, and metadata)
- * **Vector Operations:** pgvector` extension for PostgreSQL (Optional: for native similarity search)

- * **ORM (Object-Relational Mapping):** SQLAlchemy or SQLModel

3. Machine Learning & NLP Pipeline

- * **Deep Learning Framework:** PyTorch
- * **Model Hub & NLP Tooling:** Hugging Face `transformers`
- * **Vectorization/Embeddings:** `sentence-transformers` (for generating numerical representations of text)
- * **Clustering Algorithms:** `scikit-learn` (for computing cosine similarity and grouping articles)
- * **Training Infrastructure:** Google Colab / Kaggle Notebooks (for model fine-tuning)

4. Backend Application Programming Interface (API)

- * **API Framework:** FastAPI
- * **ASGI Server:** Uvicorn
- * **Data Validation:** Pydantic

5. Frontend & Data Visualization

- * **Framework:** React (Next.js)
- * **Styling:** Tailwind CSS
- * **Data Visualization:** Recharts or Chart.js (for rendering bar graphs and statistical charts)

6. Version Control & Development Tools

- * **Version Control:** Git / GitHub
- * **Environment Management:** Conda or venv

Question 19:

not getting into the details jst list of the technologies.

Answer:

- * **Python 3.10+**
- * **Scrapy**
- * **BeautifulSoup4 / lxml**
- * **PostgreSQL**
- * **SQLAlchemy / SQLModel**
- * **PyTorch**
- * **Hugging Face `transformers`**
- * **`sentence-transformers`**
- * **`scikit-learn`**
- * **FastAPI**
- * **Uvicorn**
- * **Pydantic**
- * **React / Next.js**
- * **Tailwind CSS**
- * **Recharts / Chart.js**
- * **Git / GitHub**

Question 20:

also help to understand where and how will we use the pytorch.

Answer:

In this project, you will rarely write deep, complex PyTorch code (like defining neural network layers manually). Instead, PyTorch acts as the invisible "engine" running under the hood.

Here is exactly where and how PyTorch is used in your pipeline:

1. It Powers the Hugging Face Models

When you write `pipeline("sentiment-analysis", model="...")` using Hugging Face, Hugging Face doesn't do the math. It translates your text into numbers and hands it to PyTorch.`

- * **How:** PyTorch takes those numbers (tensors) and crunches millions of matrix multiplications through the pre-trained neural network to calculate the final sentiment score.

- * **Where:** In your FastAPI backend, every time a new article is scanned for Entities (NER) or Sentiment (ABSA), PyTorch is the library executing the prediction.

2. It Powers `sentence-transformers` for Clustering

To group news articles, you need to turn the headlines into vectors (lists of numbers).

- * **How:** `sentence-transformers` uses PyTorch to pass the headline through an encoder model, outputting the 384-dimensional vector.

- * **Where:** In your backend, right after an article is scraped, PyTorch generates the embedding before it is saved to the database.

3. Hardware Acceleration (Device Management)

This is the most direct way you will write PyTorch code in your backend. You must tell the models *where* to run the math.

- * **How:** You will write initialization code using PyTorch to detect the best hardware available (CUDA for NVIDIA, MPS for Apple, or CPU as a fallback).

- * **Where:** At the startup of your FastAPI app.

```
```python
import torch
This is how you use PyTorch directly: telling the engine where to run
device = "mps" if torch.backends.mps.is_available() else "cpu"
Then you pass this 'device' to your Hugging Face models so they run fast
```
```

4. Fine-Tuning the Models (In the Cloud)

If you fine-tune the sentiment model on your custom dataset (e.g., in Google Colab), PyTorch handles the "learning."

- * **How:** PyTorch calculates the "loss" (how wrong the model's guess was) and uses an optimizer (like AdamW) to adjust the model's internal weights through backpropagation.

- * **Where:** In your Jupyter/Colab training scripts using the Hugging Face `Trainer` API (which wraps PyTorch training loops for you).

****Summary:**** You use PyTorch as the underlying mathematical engine that executes the heavy lifting for vectorization, entity extraction, sentiment prediction, and model fine-tuning.

Question 21:

that clear up things. even i was woundering the same. that we wouldn't use the pytorch directly ourselves to do math or as such.

Answer:

Exactly! You have perfectly grasped the modern ML engineering workflow.

In a modern applied AI project like this, you act as an ****AI Engineer/Architect****, not a researcher inventing new mathematics.

By using high-level libraries (Hugging Face, `sentence-transformers`) that wrap PyTorch, you get all the power of deep learning without having to manually write differential calculus or design neural network layers from scratch in PyTorch.

Your job is the architectural glue: bringing the data in via Scrapy, routing it efficiently through the PyTorch-powered models, storing the outputs correctly in PostgreSQL, and serving the results to the user via FastAPI and Next.js.

If you have any more questions about the tools, the architecture, or what specifically to include in your proposal, just let me know! Otherwise, it sounds like you are completely ready to write a stellar project proposal defense!

Question 22:

so can you update these things on the basis of or latest understandings into this context file.

Answer:

Answer:

Answer:

I have successfully updated the `PROJECT\_CONTEXT.md` file in your workspace.

Here is what I changed based on our conversation:

1. **Removed Hardware/M3 Limitations:** I updated Section 4 ("Machine Learning & Hardware Strategy") to remove the previous wording about hardware limitations. It now professionally describes PyTorch acting as the local inference engine underlying Hugging Face to process extraction, embeddings, and sentiment scoring locally without relying on external APIs.
2. **Simplified the Tech Stack List:** I replaced the descriptive paragraphs in Section 5 with the clean, direct point-by-point list of the final tools and technologies we agreed to use (Python 3.10+, Scrapy, PyTorch, FastAPI, Next.js, etc.).

Your context file is now thoroughly refined and perfectly reflects the professional, hardware-agnostic architecture we planned out!

Question 23:

can you also update about the parameters we will be working on, in the context file. wouldn't mentioning it nice?

Question 24:

can you also update about the parameters we will be working on, in the context file. wouldn't mentioning it nice?

Answer:

Answer:

I have updated the `PROJECT\_CONTEXT.md` file again.

I successfully injected a new section—**4. Key Analytical Parameters**—right between the System Architecture and the Machine Learning Strategy. It defines the four focal parameters we discussed earlier:

1. **Event Cluster ID ("The What")**
2. **Extracted Entities ("The Who")**
3. **Entity Polarity/Sentiment ("The How")**
4. **House Tag/Bias ("The Long-Term Trend")**

I also adjusted the numbering for the remaining sections so the document flows perfectly up to section 7. The context file is now extremely dense with value and covers every aspect you would need to explain the system logic out loud to your teammates or professors!

Let me know if there's any other module you'd like to dive into next!

Question 25:

- Custom Dataset: A custom dataset of approximately 500-1000 English-language Nepali political sentences will be hand-labeled.

it is true. i know that we will create custom dictionary but this handle label part? if yes, explain.

Answer:

Yes, the "hand-labeled dataset" part is absolutely true, and it is a fundamental requirement if you are doing **Transfer Learning / Fine-tuning** instead of just using an out-of-the-box model.

Here is a clear explanation of **why** you need to hand-label this dataset, **what** it actually means, and **how** you will do it.

Why Must We "Hand-Label" Data?

If you download an off-the-shelf sentiment model (like standard BERT), it was trained on Western data (movie reviews, Amazon products, US politics).

The Problem: Out-of-the-box models do not understand Nepali political context.

* **Example 1:** "The new coalition operates like a syndicate."

* **Western Model:** Might think "syndicate" is a neutral business term.

* **Nepali Context:** "Syndicate" (सिन्डिकेट) is a highly negative term implying a corrupt cartel.

* **Example 2:** "Mayor Balen brought his dozers."

* **Western Model:** Sees "dozers" (construction equipment) as neutral.

* **Nepali Context:** The "dozer policy" is highly polarized and carries strong specific sentiment.

To teach the model these specific nuances, you have to provide it with a "cheat sheet" (your custom dataset) that shows it exactly how you want it to score these specific types of sentences.

What Does "Hand-Labeling" Mean?

It simply means you and your teammates will open an Excel/CSV file with 500 to 1,000 sentences (scraped from your target news sites) and manually type the correct answer next to them. This is the "Ground Truth" you will feed into the ML model during the Colab training phase.

You will need to hand-label for two specific tasks in your pipeline:

1. Labeling for NER (Named Entity Recognition)

You need to teach the model to recognize Nepali names and parties.

* **Text:** "RSP chairman Rabi Lamichhane faced questions today."

* **Your Hand-Label:** `[ORG] RSP` chairman `[PER] Rabi Lamichhane` faced questions today.

2. Labeling for ABSA (Sentiment)

You need to teach the model the underlying sentiment toward the entity.

Your CSV file will have three columns: **Sentence**, **Entity**, and **Polarity** (Positive/Negative/Neutral).

```
* Sentence: "Despite protests, the establishment parties bulldozed the controversial bill."  
* Entity: "establishment parties"  
* Your Hand-Label: `Negative`
```

```
* Sentence: "The independent candidate surprised the traditional stronghold."  
* Entity: "independent candidate"  
* Your Hand-Label: `Positive`
```

How to do it efficiently (You don't *actually* do it 100% by hand)

Doing 1,000 rows manually takes time. In modern AI engineering, we use a trick called **Human-in-the-Loop (HITL) / Bootstrapping**.

1. **The Cheat Step:** You scrape 1,000 sentences. You run them through a powerful, free API (like Gemini via Google AI Studio or a local Llama-3 if you have access) and *ask the LLM to label them*.
2. **The Human Step:** You open the CSV file generated by the LLM. You simply read through it quickly to fix the ones the LLM got wrong (which will usually be due to the Nepali slang/context we mentioned earlier).
3. **The Result:** You now have 1,000 perfectly accurate, context-specific labels in roughly 2 hours instead of 2 days.

You take this CSV file, upload it to Google Colab, and use it to fine-tune your tiny local PyTorch model.

What to tell your professors:

If a professor asks, *"Why did you use a custom dataset?"*

You reply: *"Because pre-trained Western models misclassify Nepali socio-political terminology like 'syndicate'. We created a bootstrapped dataset of 1,000 sentences and manually vetted the ground truth labels to fine-tune the model for local context, which significantly improved accuracy."*

Does that clarify why dataset creation is a central part of the ML pipeline?

Question 26:

2. Labeling for ABSA (Sentiment)

You need to teach the model the underlying sentiment toward the entity.

Your CSV file will have three columns: Sentence, Entity, and Polarity (Positive/Negative/Neutral).

Sentence: "Despite protests, the establishment parties bulldozed the controversial bill."

Entity: "establishment parties"

Your Hand-Label: Negative

Sentence: "The independent candidate surprised the traditional stronghold."

Entity: "independent candidate"

Your Hand-Label: Positive

got it but who will select the sentence, Entity? i wana know about the creation of this file.